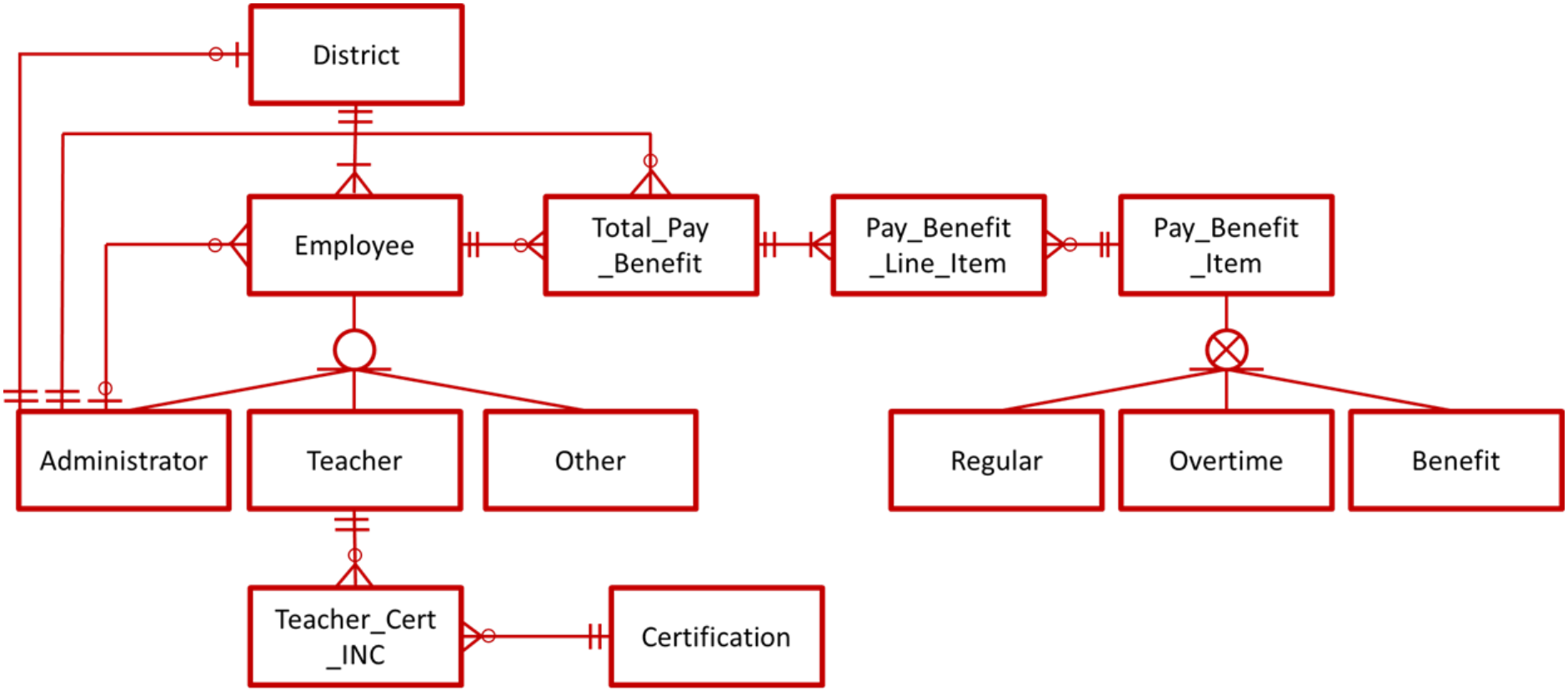
Practicum 2 Part 1: SQL-DDL

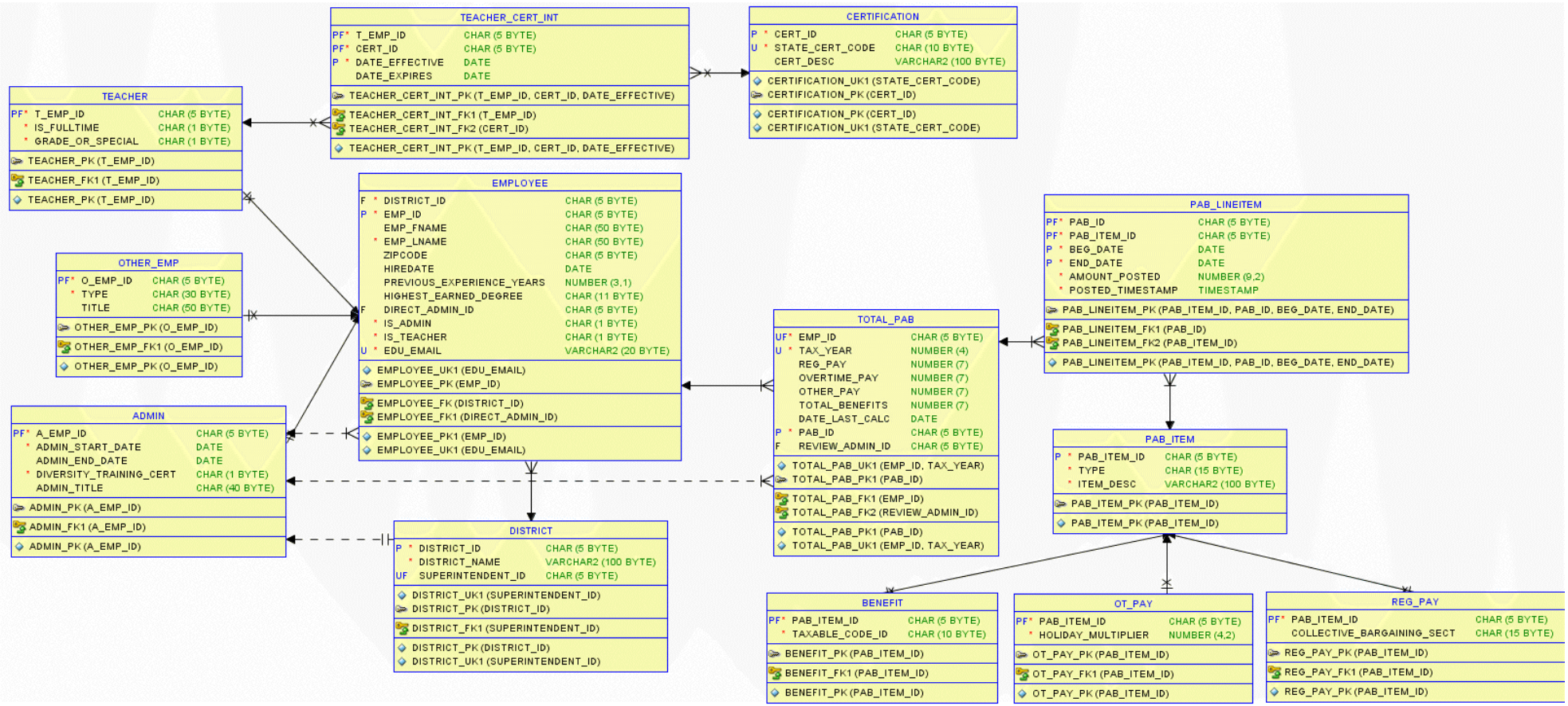
Task: Implement the database described in this documentation exactly as described below (slightly augmented from the design practicum. District and teacher certification relations were added. You will be provided the code to several relations to reduce the number of relations you need to implement. That will be shared in a separate document. The design has been frozen, make no changes, and implement what is in front of you. All table names and attribute names must be identical to what is shown below or the supplied insert statements will not work.

Conceptual Model (Entity-Relationship Diagram)[1] :



[1] The table Teacher_Cert_INC table should be Teacher_Cert_INT as shown in the next diagram

Database Design Diagram (U=Unique, P=PK, F=FK):



<u>Required Check Constraints</u> (note: primary key, foreign key, and unique constraint information are in the diagram – not null and default value information are in the tables below this section):

1. **Add a constraint to reject any inserted value for the ADMIN.ADMIN_DIVERSITY_CERT attribute other than 'Y' and 'N'.**
2. **Add a constraint to make sure that the *ADMIN.ADMIN_END_DATE* date is the same date or later than the *ADMIN.ADMIN_START_DATE* date.**
3. **[Since this constraint is a bit more involved, it will be scored equal to 2 constraints. In other words, this one counts double.] Add a constraint to check for the appearance of a valid email address (it won't check if the email is truly valid, just that it seems to be a properly formatted email address). The constraint should check to make sure all email addresses entered into EMPLOYEE.EDU_EMAIL are of the form [Anytext]@[Anytext].edu. Wherever you see [Anytext] this means that there must be *at least two characters* in that spot to be a valid input. The .edu at the end must be those exact letters but *can use any capitalization pattern*. If the rest of the email address is valid having the last four characters as '.edu', '.EDU', '.EdU', '.edU', etc., these should all be accepted. To make this case insensitive do not string together a bunch of 'OR's to test for every combination. Consider how this could be done more easily using the UPPER() function. Therefore, 'me@hh.edu', 'abrandyb@kent.eDu', 'xx@xx.EDU' would all be valid inputs. However, your constraint should reject these entries: '@.', 'me at hh.com', 'me.com@hh', 'test@test@edu', 'x@x.edu', 'a@kent.edu' (and any other examples that do not meet the stated criteria).**
4. **Constrain EMPLOYEE.HIGHEST_EARNED_DEGREE to the following values: 'GRE', 'High School', 'Associate', 'Bachelor', 'Master', and 'Doctorate'. All other entries should be rejected.**
5. **Constrain EMPLOYEE.HIREDATE to be January 1, 1950 or later (this would reject some 'impossible' entries, helping data integrity).**
6. **Constrain OTHER_EMP.TYPE to the following values: 'CUSTODIAL', 'SECURITY', 'COUNSELING', 'CONTRACT', 'LANDSCAPING', and 'UNCATEGORIZED'. All other entries should be rejected.**
7. **Constrain PAB_ITEM.TYPE to the following values: 'REGULAR PAY', 'OVERTIME PAY', 'OTHER PAY', and 'BENEFIT'. All other entries should be rejected.**
8. **Constrain TEACHER.GRADE_OR_SPECIAL to the following values: 'G' and 'S'. All other entries should be rejected.**
9. **Constrain the value for OT_PAY.HOLIDAY_MULTIPLIER to be greater than or equal to 1.00 and less than or equal to 3.50.**

Note: These constraints are representative of the *type* of constraints that would need implementing in this scenario. There are obviously many other constraints that could be written to maintain data integrity. I have not asked you to implement these others that would probably be included in a production database.

Other information from SQL Developer:

ADMIN:

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT
A_EMP_ID	CHAR(5 BYTE)	No	(null)
ADMIN_START_DATE	DATE	No	SYSDATE
ADMIN_END_DATE	DATE	Yes	(null)
DIVERSITY_TRAINING_CERT	CHAR(1 BYTE)	No	'N'
ADMIN_TITLE	CHAR(40 BYTE)	Yes	(null)

BENEFIT:

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT
PAB_ITEM_ID	CHAR(5 BYTE)	No	(null)
TAXABLE_CODE_ID	CHAR(10 BYTE)	No	(null)

CERTIFICATION:

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT
CERT_ID	CHAR(5 BYTE)	No	(null)
STATE_CERT_CODE	CHAR(10 BYTE)	No	(null)
CERT_DESC	VARCHAR2(100 BYTE)	Yes	(null)

DISTRICT:

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT
DISTRICT_ID	CHAR(5 BYTE)	No	(null)
DISTRICT_NAME	VARCHAR2(100 BYTE)	No	(null)
SUPERINTENDENT_ID	CHAR(5 BYTE)	Yes	(null)

EMPLOYEE:

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT
DISTRICT_ID	CHAR(5 BYTE)	No	(null)
EMP_ID	CHAR(5 BYTE)	No	(null)
EMP_FNAME	CHAR(50 BYTE)	Yes	(null)
EMP_LNAME	CHAR(50 BYTE)	No	(null)
ZIPCODE	CHAR(5 BYTE)	Yes	(null)
HIREDATE	DATE	Yes	SYSDATE
PREVIOUS_EXPERIENCE_YEARS	NUMBER(3,1)	Yes	0
HIGHEST_EARNED_DEGREE	CHAR(11 BYTE)	Yes	'Bachelor'
DIRECT_ADMIN_ID	CHAR(5 BYTE)	Yes	(null)
IS_ADMIN	CHAR(1 BYTE)	No	'N'
IS_TEACHER	CHAR(1 BYTE)	No	'Y'
EDU_EMAIL	VARCHAR2(20 BYTE)	No	(null)

OT_PAY:

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT
PAB_ITEM_ID	CHAR(5 BYTE)	No	(null)
HOLIDAY_MULTIPLIER	NUMBER(4,2)	No	(null)

OTHER_EMP:

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT
O_EMP_ID	CHAR(5 BYTE)	No	(null)
TYPE	CHAR(30 BYTE)	No	(null)
TITLE	CHAR(50 BYTE)	Yes	(null)

PAB_ITEM:

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT
PAB_ITEM_ID	CHAR(5 BYTE)	No	(null)
TYPE	CHAR(15 BYTE)	No	(null)
ITEM_DESC	VARCHAR2(100 BYTE)	No	(null)

PAB_LINEITEM:

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT
PAB_ID	CHAR(5 BYTE)	No	(null)
PAB_ITEM_ID	CHAR(5 BYTE)	No	(null)
BEG_DATE	DATE	No	(null)
END_DATE	DATE	No	(null)
AMOUNT_POSTED	NUMBER(9,2)	No	0
POSTED_TIMESTAMP	TIMESTAMP(6)	No	SYSDATE

REG_PAY:

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT
PAB_ITEM_ID	CHAR(5 BYTE)	No	(null)
COLLECTIVE_BARGAINING_SECT	CHAR(15 BYTE)	No	(null)

TEACHER:

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT
T_EMP_ID	CHAR(5 BYTE)	No	(null)
IS_FULLTIME	CHAR(1 BYTE)	No	'Y'
GRADE_OR_SPECIAL	CHAR(1 BYTE)	No	'G'

TEACHER_CERT_INT:

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT
T_EMP_ID	CHAR(5 BYTE)	No	(null)
CERT_ID	CHAR(5 BYTE)	No	(null)
DATE_EFFECTIVE	DATE	No	(null)
DATE_EXPIRES	DATE	Yes	(null)

TOTAL_PAY:

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT
EMP_ID	CHAR(5 BYTE)	No	(null)
TAX_YEAR	NUMBER(4,0)	No	(null)
REG_PAY	NUMBER(7,0)	Yes	(null)
OVERTIME_PAY	NUMBER(7,0)	Yes	(null)
OTHER_PAY	NUMBER(7,0)	Yes	(null)
TOTAL_BENEFITS	NUMBER(7,0)	Yes	(null)
DATE_LAST_CALC	DATE	Yes	(null)
PAB_ID	CHAR(5 BYTE)	No	(null)
REVIEW_ADMIN_ID	CHAR(5 BYTE)	Yes	(null)

CHECKLIST:

SQL DDL Practicum – 50 pts total

Checklist for submission:

- _ **Write CREATE TABLE statements to create the seven tables that are left for you to. Create these exactly as the information from the diagram states. Make no changes to this design. Your score for this section is based on how accurately you create the database from the information provided to you. Insert your create table statements** (and alter table statements if used) in the correct location of the CREATE TABLE script provided separately.

- _ **Debug your script** so that it deletes all tables in the database and then recreates them (with any changes you may have made in the statements). The provided but incomplete script is already set up for this. Just insert your statements where indicated in the script (and debug).

- _ **Retrieve the REFRESHDATA script from Blackboard (same folder). Run this script.** *Scroll back through the confirmations for inserting each row to check for errors.* If you find errors, there is a problem with your implementation that needs to be debugged. Do *not* assume any errors are the fault of the INSERT statements. If there is a problem with an insert statement it will be known quickly as it would not work for anyone. Assume these errors are due to your implementation. IMPORTANT: Do not panic if you run the data and get many errors. *The only error you should consider valid is the first error in that run of the script* (clear your query result pane between each script run so you do not become confused). Only consider the first error because many or all of the others *may* be attributed to the record failing to be inserted on the first error and other errors may be created simply because that record does not exist. Debug the first error then run the script again and find the first error again. If it is the same error then your change did not fix it, try something else. If it is a new error then this is now a valid error. Debug and repeat until no errors remain. Errors will usually be caused by misspellings of attributes or tables or caused by an improperly configured check constraint. Start debugging an error by validating these. Search the ORA-##### error message you receive (if you have errors) for more information on the specific error.

- _ Once the REFRESHDATA script inserts all the data without any errors **you are fundamentally finished except for testing and submission.** I do suggest taking the additional step of testing your implementation's check and foreign key constraints by trying to submit data that should be rejected and data that should not (if rejected it will give the error that it violates the check or foreign key constraint you are testing). Make sure inserts that *should* be rejected *are* rejected AND that inserts that *should not* be rejected *are not* rejected (test both).

- _ **Read this important information on scoring.** Your database must be implemented in *your* Oracle account! The DDL document is *confirmation* of your work, but I grade what is actually in your database, not the submitted document directly. If you have some required code in the script but not implemented in your database, you may get some partial credit for the code, but it is limited to 50% credit if not successfully implemented in Oracle. Again, make sure the database in your class Oracle database account is complete. If there is nothing implemented in Oracle, you will receive a zero for submitting *only* a script.

- _ **Confirm that your account has the fully-implemented database.** You can drop and recreate the tables fresh so that they reflect only what is in your create table script statements. Also make sure the final data is contained in your tables and has not been modified. You can run the REFRESHDATA script before submitting to ensure this. Note that if you run the CREATE TABLE script again you *must* also run the REFRESHDATA script or your tables will be empty.

- **Submit your create table script** that recreates your entire database as a text document with a .sql extension (the same format as you downloaded it). The file name should be your kent userid (In your email address, it is the part before @kent.edu) followed by _DDL. For example, a submission from me would have the filename of 'abrandyb_DDL.sql' . After you submit this file you may make no more changes to your database until it is graded and any grading questions are resolved. You need not do anything to submit what you did in the database itself, it is considered submitted when you submit the script. I will log into each account directly.

******When asking me SQL questions by email please cut and paste the entire SQL statement you are trying to run as well as the entire error message that you are receiving. Also include your Oracle login name (not your password).***